Structured Output

№ id	14
チェック	☐
機能カテゴリー	AI推論
機能概要	LLMで構造化出力を生成しJSON形式で結果を返す

Structured Output タスク取扱説明書

１．この機能でできること

テキストから情報を抽出して、きれいに整理されたデータ（JSON形式）として出力します。

できること：

- テキストから必要な情報だけを取り出す
- 名前、年齢、金額などを自動で抽出
- 決まった形式のデータに変換
- 選択肢から自動で選ばせる（分類）
- JSONとして出力

こんな場面で使えます：

- 請求書から金額や日付を自動で取り出したい
- レビューから感情（ポジティブ・ネガティブ）を判定したい
- 履歴書から名前・年齢・職歴を抽出したい
- 自由な文章を決まったデータ形式にしたい
- APIで使えるデータに変換したい

必要なもの：

- 情報を取り出したいテキスト

出てくるもの：

- きれいに整理されたデータ（JSON形式）

設定する項目：

- `system_prompt`: AIの役割（例：「請求書から情報を抽出してください」）
- `model`: 使うAIの種類
- `output_schema`: どんな形式で出力するか（項目の定義）

2．基本の使い方

使用例1: 基本的な情報抽出

やりたいこと： 人物情報を取り出す

```
# 情報を取り出したいテキスト
Input(
    text="鈴木太郎さんは35歳で、東京都在住のエンジニアです。"
)

# 設定
Params(
    system_prompt="テキストから人物情報を抽出してください",
    model="gpt-4.1-2025-04-14",
    output_schema={
        "properties": {
            "name": {"type": "string", "description": "人物
の名前"},
            "age": {"type": "integer", "description": "年
齢"},
            "location": {"type": "string", "description":
"居住地"},
            "occupation": {"type": "string", "description":
"職業"}
        },
        "required": ["name", "age"]  # 必須項目
    }
)

# 結果
```

```
Output(
    result="""{
  "name": "鈴木太郎",
  "age": 35,
  "location": "東京都",
  "occupation": "エンジニア"
}"""
)
```

使用例2: 選択肢から選ばせる（分類）

やりたいこと： レビューの感情を自動判定

```
# 判定したいテキスト
Input(
    text="このレストランの料理は素晴らしく、スタッフも親切でした。また来たいと思います。"
)

# 設定
Params(
    system_prompt="レビューを分析してください",
    model="gpt-4.1-2025-04-14",
    output_schema={
        "properties": {
            "sentiment": {
                "type": "string",
                "description": "感情分析",
                "enum": ["positive", "neutral", "negative"]
# 3つから選ぶ
            },
            "category": {
                "type": "string",
                "description": "レビューカテゴリ",
                "enum": ["food", "service", "ambiance", "value"]
            },
            "rating": {
                "type": "integer",
```

```
                "description": "評価（1-5）"
            }
        },
        "required": ["sentiment", "rating"]
    }
)

# 結果
Output(
    result="""{
  "sentiment": "positive",
  "category": "food",
  "rating": 5
}"""
)
```

使用例3: 請求書からデータ抽出

やりたいこと： 請求書から必要な情報だけを取り出す

```
# 抽出したいテキスト
Input(
    text="""
    請求書
    請求日: 2024年1月15日
    請求番号: INV-2024-001
    請求先: 株式会社ABC

    商品A: 10,000円 × 2個 = 20,000円
    商品B: 5,000円 × 1個 = 5,000円
    消費税: 2,500円
    合計: 27,500円
    """
)

# 設定
Params(
    system_prompt="請求書から情報を抽出してください",
    model="gpt-4.1-2025-04-14",
```

```
    output_schema={
        "properties": {
            "invoice_number": {"type": "string", "descripti
on": "請求番号"},
            "invoice_date": {"type": "string", "descriptio
n": "請求日"},
            "customer_name": {"type": "string", "descriptio
n": "請求先"},
            "subtotal": {"type": "integer", "description":
"小計"},
            "tax": {"type": "integer", "description": "消費
税"},
            "total": {"type": "integer", "description": "合
計金額"}
        },
        "required": ["invoice_number", "total"]
    }
)

# 結果
Output(
    result="""{
  "invoice_number": "INV-2024-001",
  "invoice_date": "2024年1月15日",
  "customer_name": "株式会社ABC",
  "subtotal": 25000,
  "tax": 2500,
  "total": 27500
}"""
)
```

使用例4: 小数も扱える

やりたいこと： 気象データを取り出す

```
# 抽出したいテキスト
Input(
    text="気温は23.5度、湿度は65%です。"
)
```

```
# 設定
Params(
    system_prompt="気象データを抽出してください",
    model="gpt-4.1-2025-04-14",
    output_schema={
        "properties": {
            "temperature": {"type": "number", "descriptio
n": "気温（摂氏）"},  # 小数対応
            "humidity": {"type": "integer", "description":
"湿度（パーセント）"}
        },
        "required": ["temperature", "humidity"]
    }
)

# 結果
Output(
    result="""{
  "temperature": 23.5,
  "humidity": 65
}"""
)
```

３．他のタスクとの組み合わせ活用と注意点

よく使う組み合わせ

1. ファイル読み取り → データ抽出

流れ： Process File → Structured Output

- **やること：** 請求書や契約書を自動でデータ化
- **使う機能：** ファイルを読む → データとして抽出

2. OCR → データ抽出

流れ： VLM OCR → Structured Output

- **やること：** スキャンした書類をデータ化

- **使う機能：** 文字読み取り → データとして抽出

3. AI処理 → データ化

流れ： LLM → Structured Output

- **やること：** 自由な文章を決まったデータにする
- **使う機能：** AIで処理 → きれいなデータに変換

4. 回答生成 → データ化

流れ： RAG → Structured Output

- **やること：** AIの回答をデータにする
- **使う機能：** 回答生成 → JSON形式に変換

5. データ抽出 → 検索登録

流れ： Structured Output → Index OpenSearch

- **やること：** 抽出したデータを検索できるようにする
- **使う機能：** データ抽出 → 検索登録

気をつけること

💡 項目の定義が重要

- `properties` で、どんな項目を取り出すか指定します
- `description` に詳しく説明を書くと、精度が上がります
- `required` で必須項目を指定できます

📊 使えるデータの種類

- `string`: 文字列（例：名前、住所）
- `integer`: 整数（例：年齢、個数）
- `number`: 小数（例：気温、金額）
- `boolean`: はい・いいえ（例：在庫あり・なし）
- `enum`: 選択肢（例：["A", "B", "C"]から選ぶ）

⚠️ 必須とオプション

- `required` に含まれる項目は必ず出力されます

- 含まれない項目は、情報がなければnull（空）になります

🔍 descriptionの重要性

- 各項目に詳しい説明を書いてください
- AIはこれをヒントにして抽出します
- 例：「請求番号（INV-で始まる番号）」

💰 高性能なAIを使う

- データ抽出には高性能なAI（GPT-4など）がおすすめ
- 複雑な項目を取り出すほど、高性能なAIが必要

⚡ enumで選択肢を制限

- カテゴリー分類などに便利
- AIは指定した選択肢の中から必ず選びます
- 例：`["positive", "neutral", "negative"]`

✅ 結果はJSON形式

- きれいに整理されたJSON形式で出力されます
- APIなどでそのまま使えます
- プログラムで扱いやすい形式です

🔄 LLMとの使い分け

Structured Output（この機能）を使うとき：

- きっちりした形式でデータを取り出したい
- APIで使えるデータがほしい
- 決まった項目を抽出したい

LLMを使うとき：

- 自由な形式で文章を生成したい
- 要約や翻訳など

⚠️ AIは完璧ではない

- 情報がテキストにない場合、推測することがあります

- 重要なデータは結果を確認することをおすすめ
- 特に金額や日付は念入りにチェック